

9-5 以 Ajax 呼叫 Controller / Action 取回 JSON 資料

前一節由 Controller 向 View 傳遞 JSON 資料，而本節則由前端以 Ajax 呼叫後端 Controller 或 API 取回資料 JSON，兩種方法有不同的優點與適用性，取決於情境和需求。接下來會談如何建立 API 服務、Ajax 命令及用 JSON 資料繪圖。

9-5-1 以 MVC 的 Controller / Action 建立 API 服務

MVC 的 Controller / Action 通常會配合 View 檢視，以輸出網頁內容；但 Controller / Action 也可單純作為 API 服務，不建立任何 View 檢視，純粹輸出 JSON 資料給前端網頁或 Ajax 程式使用。

範例 9-5 以 MVC 的 Controller / Action 建立 API 服務

請參考 JsonDataApiController.cs 控制器，用 Action 建立汽車銷售及氣溫資料 API 服務，回傳 JSON 資料給前端。

step 01 在 Helpers 資料夾建立 IUtility.cs 介面及 Utility.cs 類別程式，用來產生汽車銷售數字陣列

 Helpers/Utility.cs

```
public class Utility : IUtility
{
    //產生整數陣列,依傳入參數 num 決定產生多少個陣列元素
    public int[] GetNumbers(int num)
    {
        Random rdn = new Random(Guid.NewGuid().GetHashCode());
        int[] Nums = new int[num];
        for (int i = 0; i < num; i++)
        {
            Nums[i] = rdn.Next(1, 500);
        }
    }
}
```

```

        //或可簡化成一行
        var array = Enumerable.Range(1, num)
                                .Select(x => rnd.Next(1, 500)).ToArray();

        rnd = null;
        return array;
    }
}

```

step 02 | 在 Program.cs 中註冊 IUtility 相依性實作

Program.cs

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddTransient<IUtility, Utility>();

var app = builder.Build();

```

在 DI Container 中註冊相依性

step 03 | 新增 JsonDataApi 控制器及三個 Actions，作為 Ajax 呼叫的 API 服務，回傳 JSON 資料給前端

Controllers/JsonDataApiController.cs

```

using Microsoft.AspNetCore.Mvc;
using Mvc7_JsonWebApi.Helpers;
using Mvc7_JsonWebApi.Models;

public class JsonDataApiController : Controller
{
    private readonly IUtility _utility;
    public JsonDataApiController(IUtility utility)
    {
        _utility = utility;
    }

    //1.回傳 BMW & BENZ 汽車銷售數字 JSON
    public ActionResult<IEnumerable<CarSales>> GetCarSalesNumber()
    {
        List<CarSales> CarSalesNumber = new List<CarSales>
        {
            new CarSales { Id = 1, Car = "BMW", Salesdata = new int[] { 120,
                200, 300, 350, 400, 250, 380, 330, 500, 280, 310, 330 } },

```

以 DI 注入 IUtility 實例

回傳型別指定為 ActionResult<T>形式

```

        new CarSales { Id = 2, Car = "BENZ", Salesdata = new int[] { 220,
            150, 350, 300, 300, 200, 180, 400, 420, 210, 250, 440 } },
    };

    return CarSalesNumber; ← 將集合資料以 JSON 格式回傳
}

//2.以亂數產生 1-12 月 Audi & Lexus 汽車銷售數據
public ActionResult<IEnumerable<CarSales>> GetCarSalesNumberRandom()
{
    //以亂數產生 1-12 月數據
    var random1 = _utility.GetNumbers(12);
    var random2 = _utility.GetNumbers(12);

    List<CarSales> CarSalesNumber = new List<CarSales>
    {
        new CarSales { Id = 1438, Car = "Audi", Salesdata = random1 },
        new CarSales { Id = 9563, Car = "Lexus", Salesdata = random2 },
    };

    return CarSalesNumber; ← 將集合資料以 JSON 格式回傳
}

//3.回傳地區月均溫 JSON 資料
public ActionResult<IEnumerable<Location>> GetTemperature()
{
    //2.List 集合包含台北,台中及高雄三個地方的氣溫資料
    List<Location> Locations = new List<Location>
    {
        new Location {
            City="臺北",
            Temperature = new double[] { 16.1, 16.5, 18.5, 21.9, 25.2,
                27.7, 29.6, 29.2, 27.4, 24.5, 21.5, 17.9, 23 }
        },
        new Location {
            City="臺中",
            Temperature = new double[] { 16.6, 17.3, 19.6, 23.1, 26.0,
                27.6, 28.6, 28.3, 27.4, 25.2, 21.9, 18.1, 23.3 }
        },
        new Location {
            City="高雄",
            Temperature = new double[] { 19.3, 20.3, 22.6, 25.4, 27.5,
                28.5, 29.2, 28.7, 28.1, 26.7, 24.0, 20.6, 25.1 }
        }
    }
}

```

```

        }
    };

    return Locations; ← 將集合資料以 JSON 格式回傳
}
}

```

說明：

1. 建議用 Chrome / Firefox / Microsoft Edge 做測試（後面小節會用 Postman 測試 API），早期 IE 會將回傳資料下載成 JSON 檔，較不方便
2. 這三個 Actions 純粹扮演 API 角色，請以 F5 執行，在瀏覽器 URL 輸入以下網址測試，正常的話會顯示回傳的 JSON 資料，（Port 號碼請改成你專案為主）

```

https://localhost:7400/JsonDataApi/getCarSalesNumber
https://localhost:7400/JsonDataApi/getCarSalesNumberRandom
https://localhost:7400/JsonDataApi/getTemperature

```

9-5-2 四類簡單易用的 jQuery Ajax 指令

HTML 如需以 Ajax 呼叫遠端 API，取回 JSON 資料，可用下面四類 jQuery Ajax 指令：

```

$.ajax({
    url: apiUrl, ← 遠端 API 的 URL
    type: "POST", ← HTTP 請求方法類型
    dataType: "json", ← 指定回傳資料格式
    success: function (response) {
        ... 回呼處理程式
    }
});
    ↑
    Ajax 回呼的資料物件

$.post(url, function(response){
    ... 回呼處理程式
});
    ↑
    Ajax 回呼的資料物件

```